

Checkpoint 1 - Especificação Léxica e Analisador Léxico

Maria Rita Vieira Souza- 12321BCC024, Ana Clara Morais de Carvalho-
12311BCC048

Link para o Github do projeto: <https://github.com/mariarvsouza/GBC071-Construcao-de-Compiladores>

1 Especificação léxica da linguagem

1.1 Alfabeto de entrada

O alfabeto da linguagem desse trabalho é composto por letras (a-z, A-Z), dígitos (0-9), sublinhado ("_"), parênteses ("(", ")"), chaves ("{" , "}"), colchetes ("[" , "]"), vírgula (","), ponto e vírgula (";"), ponto ("."), aspas simples ("'"), aspas duplas ("\""), operadores aritméticos ("+", "-", "*", "/", "%") e relacionais/ lógicos ("=", "!", ">", "<", "&", "|") e os símbolos usados para os comentários ("-", ">", "<").

1.2 Case-sensitive

A linguagem desse trabalho é case-sensitive, ou seja, ela diferencia letras maiúsculas e minúsculas nas palavras reservadas e identificadores. Fizemos essa escolha para facilitar a implementação do analisador léxico.

1.3 Comentários e espaços em branco

O scanner considera como espaços em branco o espaço simples, a tabulação (`\t`) e a quebra de linha (`\n`), e apenas os ignora. Os comentários são marcados pelos caracteres abaixo, permitindo comentários de linha e de bloco, e os comentários não podem ser aninhados. Além disso, não fechar o comentário de bloco antes do fim do documento gera um erro.

Comentários de linha	->
Comentários de bloco	-> e <-

1.4 Regra de desambiguação

A linguagem adota para operadores de forma composta a regra de desambiguação "maximal munch", ou seja, o scanner sempre verifica o próximo caractere antes de decidir o token, para garantir que ele escolherá o token mais longo possível.

1.5 Palavras reservadas

A linguagem adota as palavras reservadas listadas abaixo.

Tipos de Dados		Controle de Fluxo		Estruturas de Repetição	
number	KW_NUMBER	if	KW_IF	for	KW_FOR
real	KW_REAL	else	KW_ELSE		
char	KW_CHAR	return	KW_RETURN		
string	KW_STR	break	KW_BREAK		
bool	KW_BOOL	void	KW_VOID		
		EOF	EOF		

1.6 Identificadores

São os nomes criados para identificar variáveis, funções, etc. Para a nossa linguagem, esses identificadores são case-sensitive, sem tamanho máximo definido e devem começar com uma letra ou sublinhado, seguidos por letras, dígitos ou sublinhados.

Identificadores	<code>[a-z A-Z _][a-z A-Z 0-9 _]*</code>	ID
------------------------	--	-----------

1.7 Literais

A linguagem adota os literais abaixo. No caso das strings, se ela não for fechada até o fim da linha ou do arquivo, é disparado um erro.

Literais inteiros	<code>[0-9]+</code>	INT_LIT
Literais de ponto flutuante	<code>[0-9]+ . [0-9]+</code>	FLOAT_LIT
Literais de caractere	Um único caractere diferente de ' delimitado por aspas simples	CHAR_LIT
Literais de string	Sequência de caracteres diferentes de " delimitados por aspas duplas	STR_LIT

1.8 Operadores

A linguagem adota os operadores listados abaixo.

Aritméticos	Adição	+	PLUS
	Subtração	-	MINUS
	Multiplicação	*	MULT
	Divisão	/	DIV
	Resto da divisão	%	MOD
Relacionais	Igual a	==	EQ
	Diferente de	!=	NE
	Maior que	>	GT
	Maior ou igual a	>=	GE
	Menor que	<	LT
	Menor ou igual a	<=	LE
Lógicos	E lógico	&&	AND
	OU lógico		OR
	NÃO lógico	!	NOT
Atribuição	Atribuição simples	=	ASSIGN

1.9 Delimitadores/ Pontuação

A linguagem adota os delimitadores abaixo.

()	LPAREN, RPAREN
{ }	LBRACE, RBRACE
[]	LBRACKET, RBRACKET
,	COMMA
;	SEMICOLON
.	DOT

2 Analisador léxico com o autômato finito

2.1 Estrutura da implementação

Como pedido no formato de entrega, o código fonte completo da linguagem foi entregue junto a essa documentação. Mas, de forma resumida, ele segue a seguinte estrutura:

- **TokenType:** é uma enumeração que contém todos os tipos de palavra-chave, literais, operadores, pontuações e o EOF.
- **Token:** é a classe que define o próprio token, ela contém o tipo, o lexema (texto literal), e a linha e a coluna, que são úteis para mensagens de erro.
- **LexicalError:** classe que estende a RuntimeException e define um modelo para os erros léxicos, com uma mensagem que já imprime o erro e a linha e a coluna.
- **Scanner:** é a classe que faz realmente a análise léxica do código fonte, ela armazena o código fonte, um map que relaciona as palavras reservadas com seus tokens, a

posição que o scanner está examinando no momento, a linha atual do código fonte e a coluna atual. Ela contém os métodos: `hasNext()` (verifica se ainda há caracteres não examinados no código fonte), `peek()` (olha o caractere atual sem consumir), `advance()` (consome o caractere e move a posição), `skipWhitespaceAndComments()` (consome e ignora espaços em branco e comentários) e `nextToken()` (utiliza os outros métodos para "converter" o conteúdo do código fonte em tokens).

2.2 Tratamento de erros

Os resultados dos tratamentos de erros podem ser melhores observados no arquivo separado de testes, mas a implementação é capaz de identificar os seguintes erros:

- "String não terminada": ocorre quando a string não é fechada antes do fim da linha ou do arquivo (EOF).
- "Literal de caractere inválido ou vazio": ocorre quando o caractere é vazio (""") ou quando há uma quebra de linha ou fim de arquivo logo após a abertura do caractere.
- "Literal de caractere não terminado": ocorre quando há quebra de linha, fim de arquivo ou outro caractere logo após a abertura e escrita de um caractere válido.
- "Caractere não determinado": ocorre quando existe um caractere que não se encaixa em nenhuma das regras da linguagem.
- "Comentário de bloco não terminado": ocorre quando um comentário de bloco é aberto mas o arquivo chega ao fim (EOF) antes do seu fechamento.

2.3 Documentação do autômato

Como pedido no formato de entrega, a documentação do autômato foi entregue em arquivo separado junto a essa documentação.

2.4 Testes

Como pedido no formato de entrega, os testes automatizados completos foram entregues em um arquivo separado.

Lista de testes realizados:

1. Testes com código no formato esperado, incluindo comentários de linha e de bloco e operadores compostos (para verificar, por exemplo, se o scanner reconhece "==" como "EQ" e não como um "ASSIGN" seguido de outro "ASSIGN").
2. Teste com quebra de linha no meio da string (erro "String não terminada").
3. Teste com caractere vazio (erro "Literal de caractere inválido ou vazio").

4. Teste com caractere seguido de outro caractere antes do fechamento (erro "Literal de caractere não terminado").
5. Teste com caractere inválido (erro "Caractere não determinado").
6. Teste com comentário de bloco não fechado (erro "Comentário de bloco não terminado").

3 Log do uso de IA

Como pedido no formato de entrega, o log do uso de IA foi entregue em arquivo separado junto a essa documentação.